

Client Requirement for Automated Sentiment Analysis System

Project Deadline:

- The project must be completed **within 7 days** from the assignment start date.

Project Overview: This project will focus on automating sentiment analysis for client feedback and reviews. The system will use a pre-trained model for sentiment classification (positive, neutral, or negative) and will generate actionable insights based on customer feedback. A robot (automation system) will handle and process new reviews as they come in, offering businesses the ability to quickly assess customer sentiment and make informed decisions.

Project Requirements:

1. Dataset Handling:

- Collect a dataset consisting of **client feedbacks** or **reviews** (CSV or JSON format).
- Reviews should contain textual content along with a sentiment label (positive, neutral, negative).
- The system should be able to handle incoming reviews automatically, either by importing new data from a file or an API.

2. Sentiment Analysis Automation:

- **Pre-trained Sentiment Classifier:** The system should use a pre-trained **Naive Bayes classifier** or any suitable model for sentiment analysis, and automatically classify the sentiment of each incoming review.
- **Sentiment Classification:** Reviews should be classified as **Positive, Neutral, or Negative**, and the system should update the review status with the sentiment result in the dataset or a dashboard.
- **Automation:** Once a new review is received, the robot should process the review and update the sentiment status without manual intervention.

3. Model Training and Adaptation:

- The system should allow the classifier to be retrained periodically with new data to improve accuracy and adapt to changing customer sentiment trends.
- **Preprocessing Pipeline:** Automatically clean and preprocess text reviews:
 - Remove HTML tags, special characters, and stopwords.
 - Tokenize, lemmatize, and lowercase the reviews for better processing.
- Use **scikit-learn** for model training and **NLTK** for preprocessing the text data.

4. User Interface for Displaying Results:

- Develop a **dashboard or reporting interface** that allows the business to view the processed reviews and their sentiment results in real time.
- The interface should show:
 - The review text.
 - The sentiment classification result (Positive, Neutral, Negative).
 - Sentiment distribution statistics (e.g., how many reviews are positive, neutral, and negative).
 - A graphical representation (e.g., pie charts or bar graphs) of the sentiment analysis.

5. Automation Process:

- **Incoming Data Handling:** The system must be able to automatically process incoming reviews either through:
 - A file upload system (CSV/JSON).
 - API integration to collect real-time reviews from client platforms.
- **Review Updates:** The system should be capable of:
 - Automatically detecting newly uploaded reviews.
 - Processing and analyzing each review's sentiment.
 - Updating the corresponding status and dashboard.
- **Email/Notification Alerts:** Set up automated notifications for the business when there is a large number of **negative reviews**.

6. Performance Monitoring:

- Ensure that the automation system is capable of handling multiple client reviews per minute without lag.
- The system should be able to scale up to handle **high volumes of reviews** efficiently.

7. Final Submission Requirements:

- The complete code, including:
 - **Review preprocessor scripts** (for cleaning and preprocessing).
 - **Model training and adaptation scripts** (for sentiment analysis).
 - **Dashboard or UI code** for displaying sentiment results.

- **Automation scripts** for handling incoming reviews and updating statuses.
- **Pre-trained model file** (sentiment_model.pkl) and **vectorizer file** (vectorizer.pkl).
- **Documentation** explaining:
 - The approach used to build the automation system.
 - How to retrain the model with new data.
 - Instructions on setting up and deploying the automation pipeline.
 - Any limitations or potential improvements for future scalability.

8. Evaluation Criteria:

- **Accuracy:** The sentiment analysis model should provide accurate classifications of reviews as positive, neutral, or negative.
- **Real-time Processing:** The system should automatically process incoming reviews and display results without delays.
- **Scalability:** The system must be scalable to accommodate a growing volume of reviews.
- **User Experience:** The reporting dashboard should be intuitive, and the business should be able to easily interpret sentiment analysis results and trends.

Penalties:

- Failure to meet the deadline will result in **deduction of points** or **reduced payment** based on the severity of the delay.
- Incomplete functionality or failure to meet the automation requirements will lead to **project rejection**.

Project: Sentiment Analysis GUI with Tkinter – Comprehensive Guide

Overview

Sentiment analysis is a fundamental application of Natural Language Processing (NLP), where we analyze text to determine the underlying sentiment (positive, neutral, or negative). This project focuses on building a **Sentiment Analysis Application with GUI** using **Tkinter** in Python. By using a pre-trained model, we classify customer reviews and display the sentiment with corresponding emojis.

Why is this Important in Data Science?

1. **Real-World Application:** Businesses can analyze customer reviews to improve products or services.
2. **Text Data Insights:** Provides insights into customer satisfaction, which can guide decision-making.
3. **Learning NLP:** Introduces students to core NLP concepts like text preprocessing, model training, and sentiment analysis.

Tools and Libraries Used

1. **Python:** Programming language for writing the project.
2. **Tkinter:** GUI framework for Python to create a user interface.
3. **scikit-learn:** Machine learning library used to train the sentiment classifier.
4. **NLTK:** Natural Language Toolkit for text preprocessing (tokenization, stopwords, lemmatization).
5. **pandas:** For data manipulation and loading the dataset.
6. **matplotlib:** To display emojis in the Tkinter window.
7. **Pillow:** To manage image loading and manipulation.
8. **Pickle:** For saving and loading the trained machine learning model and vectorizer.

Project Steps

1. Set Up the Environment

- Install Python.
- Install the required libraries:

```
pip install scikit-learn pandas nltk matplotlib pillow tk
```

2. Dataset

The dataset (customer_reviews.csv) contains customer reviews. It is used to train a sentiment classifier. Ensure the CSV contains a column like Detailed Review. You can make by the CSV by yourself or download from Keggale.

3. Preprocessing

Text preprocessing is essential to clean the data before feeding it into the model. We:

- Remove HTML tags.
- Remove special characters.
- Lowercase the text.
- Tokenize the text.
- Lemmatize words (convert to base form).
- Remove stopwords (common words like "is", "the").

Code for Preprocessing:

```
import re

from nltk.corpus import stopwords
from nltk.stem.wordnet import WordNetLemmatizer

def preprocess_review(text):
    text = re.sub('<.*?>', '', text)
    text = re.sub(r'[\^\\w\\s]', '', text)
    text = text.lower()
    text = text.split()

    lmtzr = WordNetLemmatizer()

    text = [lmtzr.lemmatize(word, 'v') for word in text if
word not in set(stopwords.words('english'))]

    return " ".join(text)
```

4. Model Training



We use a **Naive Bayes Classifier** from scikit-learn for text classification. The model is trained on preprocessed reviews and saved for future use.

Code for Model Training:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import GaussianNB
import pickle

# Vectorize the text data (Bag of Words)
cv = CountVectorizer(max_features=5000, ngram_range=(1,3))
X = cv.fit_transform(preprocessed_reviews).toarray()
y = [1 if i % 2 == 0 else 0 for i in range(len(X))] # Dummy
binary classification

# Train the Naive Bayes model
classifier = GaussianNB()
classifier.fit(X, y)

# Save the model and vectorizer
with open('sentiment_model.pkl', 'wb') as model_file:
    pickle.dump(classifier, model_file)

with open('vectorizer.pkl', 'wb') as vec_file:
    pickle.dump(cv, vec_file)
```

5. GUI Development

Using **Tkinter**, we build a graphical user interface (GUI) to display a review, predict the sentiment, and show an emoji representing the sentiment. A button allows switching between different reviews.

Code for the Tkinter GUI:

```
import tkinter as tk
from tkinter import ttk
from PIL import Image, ImageTk
import pandas as pd
import pickle

# Load the model and vectorizer
with open('sentiment_model.pkl', 'rb') as model_file:
    model = pickle.load(model_file)
with open('vectorizer.pkl', 'rb') as vec_file:
    vectorizer = pickle.load(vec_file)

# Load dataset
reviews = pd.read_csv('customer_reviews.csv', encoding='ISO-8859-1')

# Function to update review and emoji
def update_feedback(index):
    review_text = reviews['Detailed Review'].iloc[index]
    processed_review = preprocess_review(review_text)
    X_review =
vectorizer.transform([processed_review]).toarray()
    predicted_sentiment = model.predict(X_review)[0]
```

```
emoji_map = {1: happy_emoji, 0: neutral_emoji, -1:
angry_emoji}

emoji_image = emoji_map.get(predicted_sentiment,
neutral_emoji)

emoji_label.config(image=emoji_image)
emoji_label.image = emoji_image
review_label.config(text=review_text)

# Initialize Tkinter window
root = tk.Tk()
root.title("Sentiment Analysis")

# Load emojis
happy_emoji =
ImageTk.PhotoImage(Image.open('happy_emoji.png').resize((100,
100)))

neutral_emoji =
ImageTk.PhotoImage(Image.open('neutral_emoji.png').resize((100
, 100)))

angry_emoji =
ImageTk.PhotoImage(Image.open('angry_emoji.png').resize((100,
100)))

# Set up the labels and buttons
emoji_label = tk.Label(root)
emoji_label.grid(row=0, column=0, padx=10, pady=10)

review_label = tk.Label(root, wraplength=300,
justify="center")
review_label.grid(row=1, column=0, padx=10, pady=10)
```

```
current_index = [0]

def next_review():
    current_index[0] = (current_index[0] + 1) % len(reviews)
    update_feedback(current_index[0])

next_button = ttk.Button(root, text="Next Review",
command=next_review)

next_button.grid(row=2, column=0, padx=10, pady=10)

# Initialize with the first review
update_feedback(current_index[0])
root.mainloop()
```

Project Summary

- **Tools Used:** Python, Tkinter, scikit-learn, NLTK, pandas, matplotlib.
- **Steps:** Data preprocessing, model training, GUI creation, and real-time sentiment analysis with emoji representation.
- **Outcome:** A fully functioning GUI where users can switch between customer reviews and see their sentiment with appropriate emojis.

Tough Questions for Students

1. **Why did we choose the Naive Bayes classifier for this sentiment analysis project? What makes it suitable for text classification?**
2. **Explain the significance of preprocessing steps like lemmatization, stopwords removal, and tokenization. How would the model's performance change without these steps?**
3. **What role does CountVectorizer play in the project? How would the results differ if we used TfidfVectorizer instead?**

4. How does the Bag-of-Words model handle unseen words in the test data that were not present in the training data?
5. Discuss the limitations of using n-grams in text vectorization. How do n-grams impact model accuracy and computational complexity?
6. What are the key differences between stemming and lemmatization? Why did we use lemmatization in this project?
7. What challenges arise when handling imbalanced datasets in sentiment analysis? How could you modify this project if the dataset were heavily imbalanced?
8. Explain why we used pickle to save the model and vectorizer. What are the alternatives, and how do they compare in terms of performance and usability?
9. Why is it important to split the dataset into training and test sets? What could happen if we trained the model on the entire dataset without testing?
10. How does Tkinter interact with machine learning models in this project? What happens behind the scenes when a new review is selected?
11. In what scenarios might the sentiment prediction in this project be incorrect? What would you do to improve accuracy?
12. If you had to scale this project for a real-time application with thousands of users, what architectural changes would you recommend?
13. How would you extend this project to handle more nuanced sentiments beyond just positive, neutral, and negative (e.g., very positive, very negative)?
14. How does the Naive Bayes algorithm make predictions based on probabilities? What assumptions does it make, and how do they affect its performance on real-world data?
15. Explain the purpose of the confusion matrix and classification report in this project. What can these metrics tell us about the model's performance that accuracy alone cannot?

Note to Students

If you completed the project by yourself, you should be able to answer these questions confidently. These questions are designed to assess your understanding of the concepts. Do not seek help from others; rely on your own knowledge and the work you've done.

